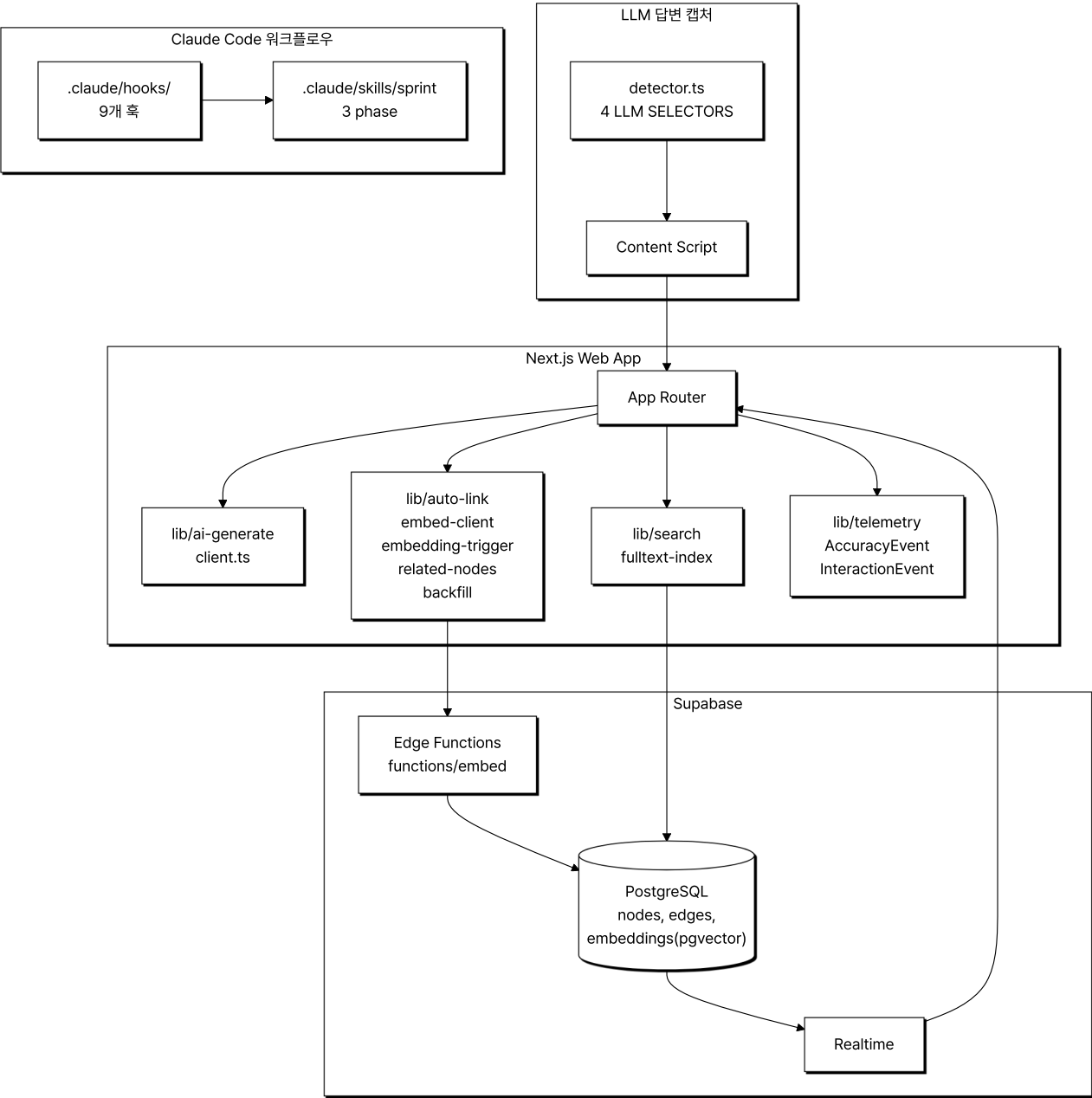


Portfolio

[MINDGRAPH] - AI 지식 캡처 & 그래프 시각화

ChatGPT·Gemini·Claude·Grok 4개 LLM 서비스의 답변을 hostname 자동 감지로 캡처하고, 캡처한 응답을 임베딩해 관련 노드를 추천·풀텍스트 검색·정량 평가까지 한 시스템 안에서 다루는 LLM 데이터 도구입니다. Claude Code 위에 9개 혹(hocs)과 plan·qa·ship 3 phase /sprint 워크플로우, 5층 컨텍스트 영속화를 직접 설계해 에이전트가 잘 동작하도록 환경 자체를 코드화한 컨텍스트 엔지니어링·하네스 엔지니어링 사례이며, 도메인 getmindgraph.com 등록 후 출시 전 1인 프로토타입 단계입니다.

전체적인 아키텍처



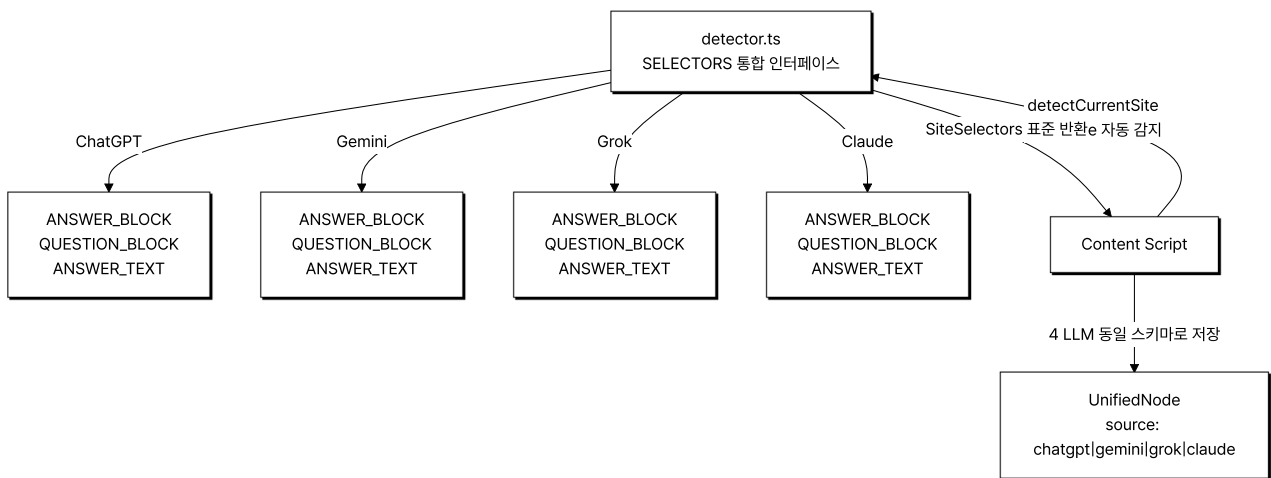
- **Architecture:** LLM 답변 캡처(Extension), 임베딩 생성(Supabase Edge Function), 추천·검색(auto-link, search), 정량 평가(telemetry), AI 협업 운영(Claude Code 9 혹:3 phase sprint)이 한 시스템 안에서 연결됩니다.

Case 1. 4개 LLM 서비스 통합 표면을 단일 지점으로 추상화

1. 문제 원인

- ChatGPT(OpenAI)·Gemini(Google)·Grok(xAI)·Claude(Anthropic) 4개 LLM 서비스가 각자 다른 응답 DOM 구조를 가지고 있고 사전 공지 없이 자주 바뀌어, 4개 서비스의 응답을 한 시스템에서 통합 캡처하려면 표준 인터페이스로 묶는 통합 레이어가 필요했습니다.
- 초기에는 서비스별 선택자가 여러 파일에 분산되어 한 서비스 DOM 변경이 다수 파일 수정으로 이어졌고, 새 LLM 서비스를 추가하는 비용도 함께 늘었습니다.

2. 해결 과정



- **선택자 통합 인터페이스:** 4개 LLM 서비스의 답변 영역 DOM 선택자를 **detector.ts**의 **SELECTORS** 상수에 **ANSWER_BLOCK · QUESTION_BLOCK · ANSWER_TEXT** 표준 키로 모아, 서비스별 차이를 단일 인터페이스로 추상화했습니다.
- **hostname 자동 감지:** **detectCurrentSite()**가 **window.location.hostname** 기반으로 현재 LLM 서비스를 식별해 올바른 선택자를 반환합니다. 사용자가 서비스를 수동 선택할 필요가 없습니다.
- **UnifiedNode source 필드:** 캡처한 답변은 **source: 'chatgpt' | 'gemini' | 'grok' | 'claude'** 필드를 포함해 동일 스키마로 저장되어, 이후 추천·검색·평가가 LLM 서비스 구분 없이 동작합니다.
- **새 LLM 서비스 추가 경로:** SELECTORS 상수에 새 항목을 추가하면 Content Script·UnifiedNode·임베딩·검색·추천이 모두 그대로 동작하는 단방향 확장 구조를 유지했습니다.

3. 결과

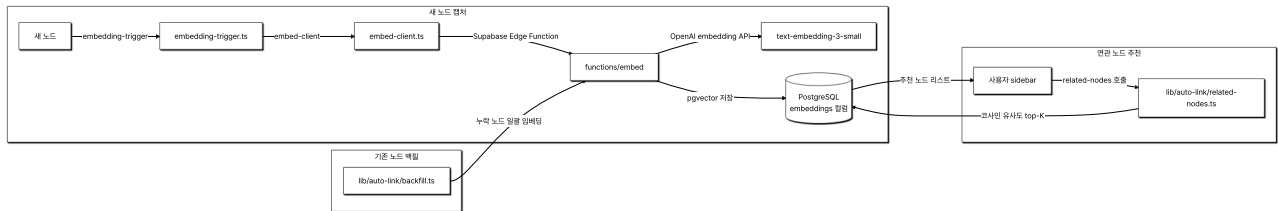
- **성과:** 4개 LLM 서비스의 응답 DOM 변경에 대응할 때 수정 범위가 **detector.ts** 상수 한 곳으로 한정되었고, 새 LLM 서비스를 추가할 때도 SELECTORS 등록 하나로 시스템 전체가 통합되도록 LLM 통합 표면을 단일화했습니다.
- **배운 점:** 다중 LLM 통합 시스템에서는 각 서비스 응답 표현(DOM) 차이를 한 곳에 모아 표준 인터페이스로 추상화하는 통합 레이어 설계가 핵심이고, SELECTORS 상수 한 곳 수정만으로 4개 LLM 서비스 응답 DOM 변경에 대응할 수 있게 되었습니다.

Case 2. 임베딩 기반 연관 노드 추천 시스템 설계

1. 문제 원인

- 캡처한 LLM 답변이 누적될수록 사용자가 관련 답변을 다시 찾기 어려워졌고, 단순 키워드 매칭으로는 의미가 비슷한 답변(다른 LLM이 다른 표현으로 답한 같은 주제)을 추천할 방법이 없었습니다.

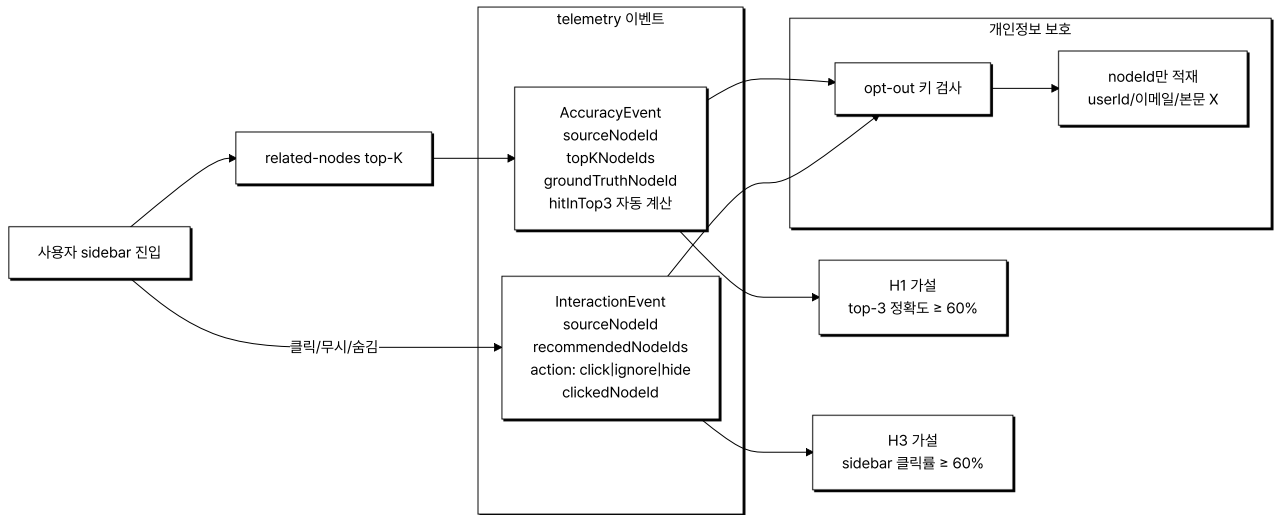
- 사용자가 sidebar에서 한 노드를 보고 있을 때 관련 노드를 측삭에서 보여주는 사용자 흐름을 위해서는 추천 검색이 화면 지연 없이 작동해야 했습니다.
- 새 노드 캡처마다 임베딩을 생성하면 비용이 누적되어, 임베딩 생성 트리거와 백필을 분리하는 설계가 필요했습니다.



3. 결과

Case 3. 추천 정확도·클릭률 정량 측정 인프라 사전 구축

2. 해결 과정



- **AccuracyEvent:** lib/telemetry.ts 의 **AccuracyEvent** 가 추천 결과(**topKNodeIds**)에 정답 노드(**groundTruthNodeId**)가 포함되는지 **hitInTop3** 필드로 자동 계산해 top-3 정확도를 측정합니다.
- **InteractionEvent:** **InteractionEvent** 가 추천 노드별 **click · ignore · hide** 행동을 기록해 sidebar 클릭률을 산출합니다.
- **임계 가설 명시:** H1 가설은 "top-3 정확도 ≥ 60%", H3 가설은 "sidebar 클릭률 ≥ 60%"로 코드 주석에 명시되어 출시 후 임계 충족 여부를 추적할 수 있도록 사전 설계했습니다.
- **저장소 이원화:** Extension은 **chrome.storage.local** 에, 웹앱은 **window.localStorage** 에 fallback 저장해 두 환경에서 동일 이벤트 스키마를 유지합니다.
- **개인정보 보호 설계:** telemetry는 **nodId** 만 기록하고 **userId ·이메일·본문**은 적재하지 않으며 opt-out 키가 있으면 no-op으로 동작합니다.

3. 결과

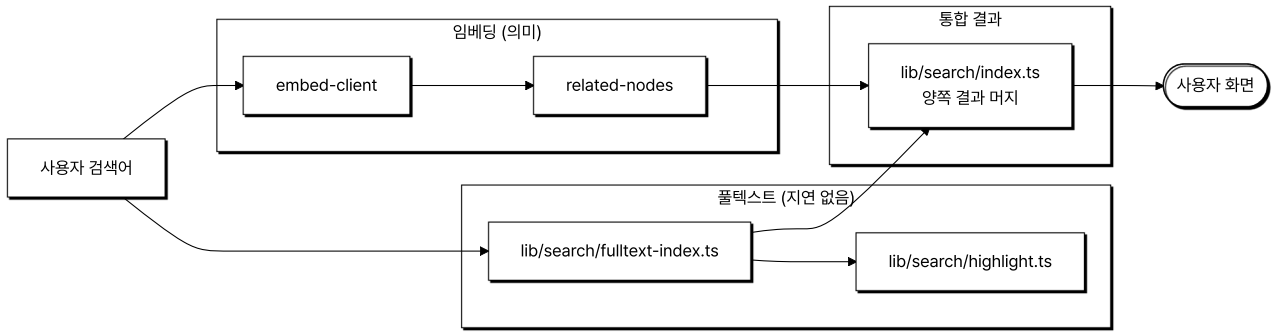
- **성과:** 추천 정확도(top-3 ≥ 60%)·sidebar 클릭률(≥ 60%) 두 가지 임계를 H1·H3 가설로 명시하고 코드로 측정하는 정량 검증 인프라를 출시 전에 사전 구축했습니다.
- **배운 점:** AccuracyEvent·InteractionEvent와 opt-out·nodId 한정 적재 정책을 출시 전 같은 코드 경로에 함께 넣어, 출시 직후부터 top-3 정확도·sidebar 클릭률을 측정할 수 있는 상태로 만들었습니다.

Case 4. 풀텍스트 검색과 임베딩 검색 통합 사용자 흐름

1. 문제 원인

- 사용자가 노드를 찾는 방식이 두 가지로 갈렸습니다. 키워드를 정확히 기억하는 경우(풀텍스트)와 의미만 기억하는 경우(임베딩). 한쪽만 지원하면 사용자가 매번 다른 도구로 옮겨가야 했습니다.
- 풀텍스트 인덱스는 지연 없이 결과를 반환할 수 있지만 임베딩 검색은 외부 API 호출 비용이 있어, 두 검색을 어떻게 묶을지에 대한 결정이 필요했습니다.
- 검색 결과 하이라이트가 없으면 어떤 부분이 매칭됐는지 사용자가 알 수 없어 검색 신뢰도가 떨어졌습니다.

2. 해결 과정



- **풀텍스트 인덱스:** `lib/search/fulltext-index.ts` 가 노드 본문을 토큰화해 인메모리 인덱스로 관리하고, 사용자 타이핑 직후 결과를 반환합니다.
- **하이라이트:** `lib/search/highlight.ts` 가 검색어와 매칭된 부분을 강조 표시해 사용자가 매칭 위치를 바로 확인하게 합니다.
- **임베딩 검색 연계:** Case 2의 `related-nodes` 를 검색 경로에서도 호출해 의미 유사 노드를 함께 가져옵니다.
- **결과 통합:** `lib/search/index.ts` 가 풀텍스트 결과와 임베딩 결과를 머지해 사용자에게 한 화면으로 보여줍니다.
- **호출 비용 분리:** 풀텍스트는 입력 직후, 임베딩 검색은 일정 지연 후 호출하도록 분리해 외부 API 호출 비용을 통제했습니다.

3. 결과

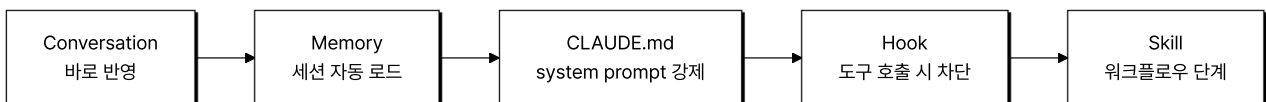
- **성과:** 키워드 검색(지연 없음)과 의미 검색(임베딩)을 한 사용자 흐름에서 처리하면서 외부 API 호출 비용은 임베딩 경로로 한정했고, 하이라이트로 검색 신뢰도를 확보했습니다.
- **배운 점:** 풀텍스트는 입력 직후, 임베딩 검색은 일정 지연 후에 호출하도록 분리해 외부 API 호출 비용을 통제하면서 사용자 화면에는 두 결과를 머지해 노출했습니다.

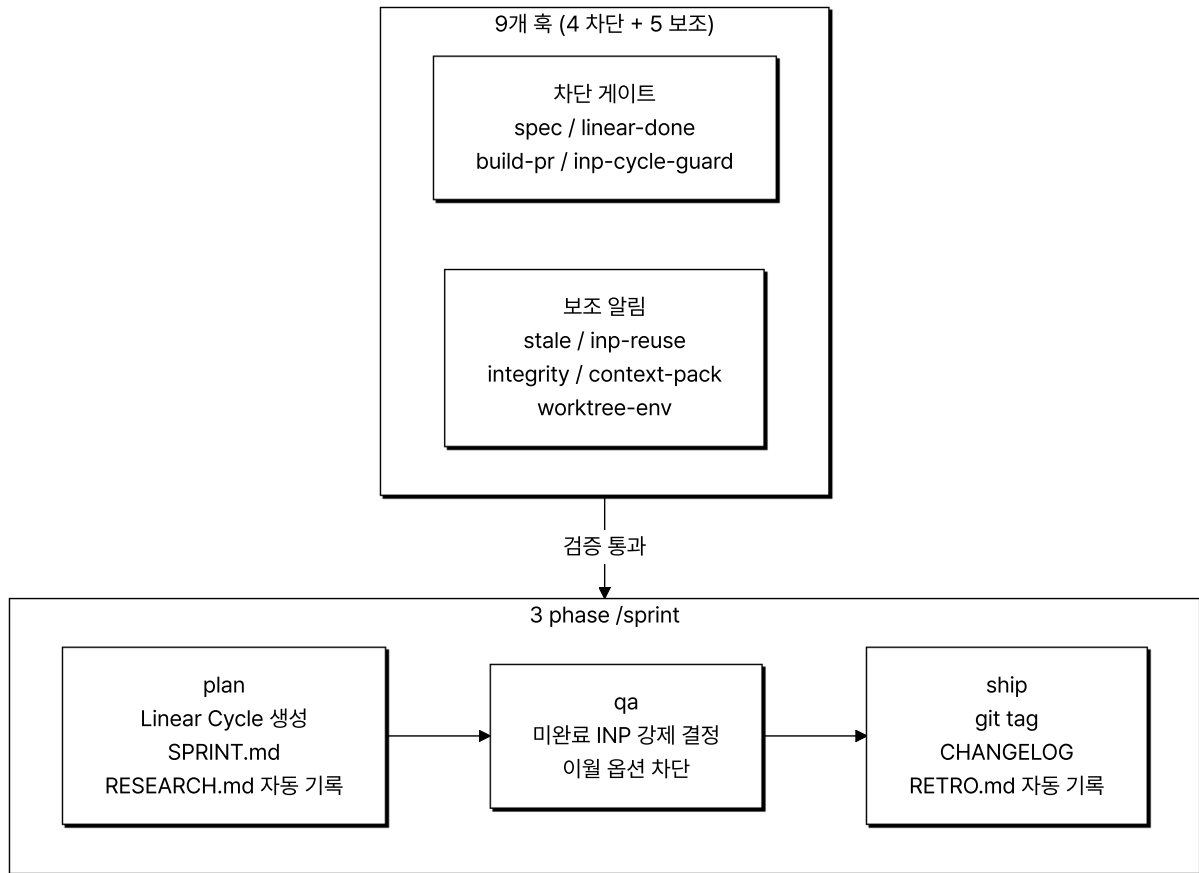
Case 5. 컨텍스트 엔지니어링·하네스 엔지니어링: Claude Code 위 9 혹(하네스) + 3 phase sprint + 5층 컨텍스트 영속화

1. 문제 원인

- Claude Code를 도구로만 쓰면 세션 간 컨텍스트가 초기화되어 같은 규칙·교정을 LLM에게 반복 설명하는 프롬프트 비용이 누적되었고, 이는 LLM 활용 효율을 결정하는 본질 문제였습니다.
- AI가 사용자 확인 없이 commit 생성·이슈 완료 처리·cycle 이월 같은 자율 행동을 해서, LLM 출력 검증을 시스템 수준에서 강제하지 않으면 잘못된 출력이 그대로 main 브랜치에 머지될 위험이 있었습니다.
- 코드·이슈·문서·배포 기록 네 시스템이 같은 정보를 중복 관리해, LLM이 현재 상태를 정확히 파악 못 한 채 응답을 생성하는 컨텍스트 부족 문제가 누적되었습니다.

2. 해결 과정





- **5층 컨텍스트 영속화:** Conversation-Memory-CLAUDE.md-Hook-Skill 5층에 규칙을 분산해, 교정이 바로 반영되고 다음 세션에 자동 로드되며 매 세션 system prompt에 강제되고 도구 호출 시 차단되며 워크플로우 단계로 구조화되도록 했습니다. 이는 LLM 컨텍스트 엔지니어링을 코드 자산으로 정착시킨 사례입니다.
- **9개 혹은 검증 게이트:** 차단 게이트 4개(`spec-gate.sh` 로 스펙 미승인 시 코드 작성 차단, `linear-done-gate.sh` 로 사용자 승인 전 이슈 완료 차단, `build-pr-block.sh` 로 워커 브랜치 PR 차단, `inp-cycle-guard.sh` 로 active cycle 외 이슈 cycle 이동 차단)와 보조 알림 5개(`stale-warn.sh` · `inp-reuse-suggest.sh` · `integrity-sync.sh` · `context-pack-stale.sh` · `worktree-env-symlink.sh`)로 분류해 LLM 출력 검증을 시스템 수준에서 강제합니다.
- **/sprint 메타 스킬 3 phase:** plan(Linear Cycle 생성·SPRINT.md 작성·RESEARCH.md 자동 기록)·qa(미완료 INP를 완료 또는 archive로 강제, 이월 옵션 mechanism 차단)·ship(git tag·CHANGELOG·RETRO.md 자동 기록)을 단일 명령으로 연결하고, 각 phase 진입 전 이전 산출물 존재 여부를 검증해 LLM에 일관된 컨텍스트가 단계별로 주어지도록 강제합니다.
- **4중 SSOT:** SPRINT.md·Linear Cycle·Git·CHANGELOG 네 시스템에 역할 하나씩만 부여해 LLM이 항상 단일 진실 원천에서 컨텍스트를 받도록 했습니다.
- **부서·에이전트 분리 + context-map 자동 라우팅:** 7개 부서(design·dev·docs·marketing·ops·product·qa)와 9개 에이전트(ceo·frontend/backend/qa-engineer·product-manager·ui-ux-designer·marketing-strategist·ops-engineer·knowledge-logger)별로 `department/{팀}/CLAUDE.md` 와 `docs/ lifecycle` 폴더(active/playbook/artifacts/drafts/archive)를 분리하고, `RULES/context-map.md` 정책 doc + `SYSTEM/schemas/code-doc-mapping.yaml` 매핑 데이터로 task_type 6종(new_feature·ui_change·api_change·copy_change·bug_fix·refactor) 및 코드 path glob(예: `storage/**` 변경 시 `DATABASE_SCHEMA.md` · `BACKEND.md` · `STATE_MANAGEMENT.md`)을 must-read doc에 자동 매핑했습니다. CEO 에이전트가 워커(frontend-engineer·backend-engineer·qa-engineer)에 위임할 때 task_types.must_read + path_mappings 합집합만 prompt에 첨부되어, 전 부서 docs 끌어오기를 5개 내외의 doc 첨부로 줄였습니다. `scripts/docs/validate.ts` 의 `validateContextMap()` 이 must-read 실재성을 검증해 라우팅이 깨지지 않도록 했습니다.

3. 결과

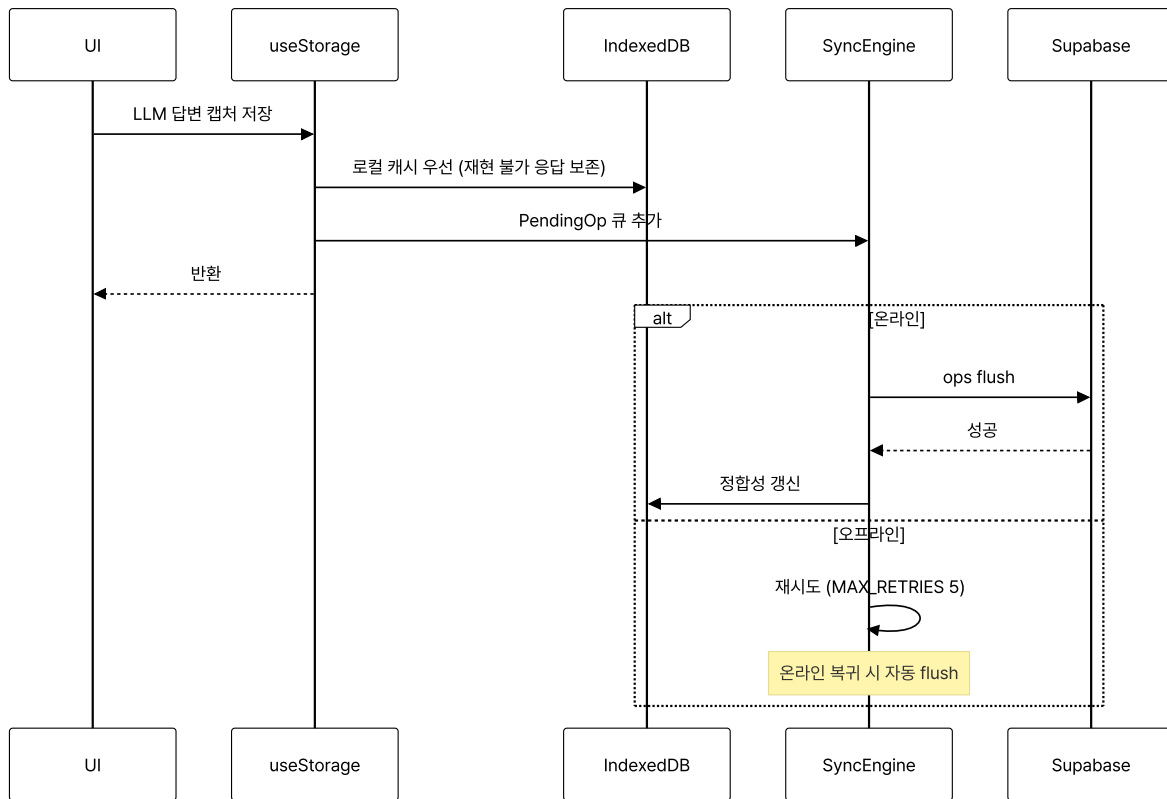
- **성과:** 이슈 등록·SPRINT.md·CHANGELOG·Git 태그·Linear Done 같은 sprint 라이프사이클 반복 작업을 9개 혹 `/sprint` 3 phase 로 자동화해 LLM이 정확한 컨텍스트를 받아 검증된 출력을 만드는 운영 사이클을 코드화했습니다.
- **배운 점:** Conversation·Memory·CLAUDE.md·Hook·Skill 5층과 9개 혹 `/sprint` 3 phase·4종 SSOT를 코드로 두면서, sprint 라이프사이클 반복 작업과 검증 게이트가 사용자 확인 없이 자율 실행되는 위험을 같은 코드 경로에서 처리하게 되었습니다.

Case 6. LLM 응답 데이터 무손실 동기 파이프라인

1. 문제 원인

- LLM 답변은 모델 응답이라 같은 프롬프트로도 다시 동일하게 재현되지 않을 수 있어, 한 번 캡처한 결과를 잃지 않는 무손실 저장 정책이 LLM 데이터 파이프라인에 필수였습니다.
- LLM 답변이 네트워크 단절로 유실되면 사용자가 다시 LLM에게 같은 프롬프트를 던져 응답을 받아야 했고, 이는 LLM 호출 비용과 사용자 시간을 동시에 낭비하는 데이터 무결성 문제였습니다.
- Supabase RLS 정책 적용 중 오프라인 상태 인증 토큰 만료 변수까지 함께 처리해야 해, 단순 캐시가 아닌 시스템 결정이 필요했습니다.

2. 해결 과정



- **로컬 캐시 우선 저장:** 캡처한 LLM 답변을 IndexedDB에 먼저 쓰는 Write-Behind 방식으로 LLM 호출과 저장을 분리해 재현 불가능한 응답을 네트워크 상태와 무관하게 보존합니다.
- **Pending Ops 큐:** `sync-engine.ts` 의 `addPendingOps` / `flushPendingOps` 가 동기화 실패를 큐로 흡수하고, 온라인 복귀 시 자동으로 flush됩니다.
- **재시도 한도:** `MAX_RETRIES = 5` 상수로 무한 재시도를 방지하고 초과 시 `status: 'failed'` 로 마킹해 재시도 비용 폭발을 차단했습니다.
- **Realtime 통합:** Supabase Realtime이 다른 디바이스에서 캡처된 LLM 답변을 현재 디바이스 캐시에 자동 반영합니다.
- **캐시 분리:** 클라우드 캐시(`mindgraph-cloud-cache`)와 로컬 데이터(`mindgraph-web`)를 별도 IndexedDB로 두어 로그아웃 시 클라우드만 비우고 비로그인 사용자 LLM 답변은 로컬에 유지됩니다.

3. 결과

- **성과:** 오프라인 상태에서 LLM 답변 캡처가 동작하고 온라인 복귀 시 큐가 자동 flush되며, 같은 LLM에게 같은 프롬프트로 재요청해야 하는 비용·시간 손실이 사라졌습니다.
- **배운 점:** IndexedDB 우선 저장·Pending Ops 큐(MAX_RETRIES=5)·Supabase Realtime·로컬/클라우드 캐시 분리를 묶어, 오프라인 캡처와 온라인 복귀 자동 flush, 다른 디바이스 변경의 자동 반영을 같은 동기 파이프라인에서 처리했습니다.